Slide 1/10

Java Interview Prep

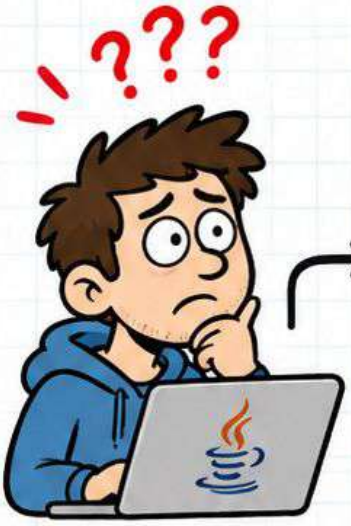
10 CompletableFuture INTERVIEW TRAPS

Async Code ≠ Faster Code!

spring boot API Latency (Production)

BEFORE: 200ms ✓ → AFTER: 8s ⚠





Java API (Spring Boot)

Thread Pool (Executor)

- Worker-1 BLOCKED
- Worker-2 BLOCKED
- Worker-3 BLOCKED
- ...
- Worker-N BLOCKED

Task Queue (Growing...)

- Task #1024
- Task #1025
- Task #1026
- ...
- Task #10987

Why did async make production SLOWER?

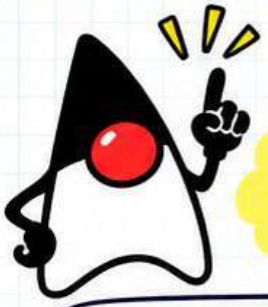
SENIOR JAVA DEVELOPER GUIDE

Blocking • Executors • Exceptions • Timeouts • Fan-out

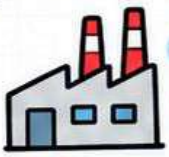
Save this for your interview! Follow @java_interview_prep

Slide 2/10

Java Interview Prep



MISTAKE #1: BLOCKING WITH join()



Production Scenario:

Order API starts an async customer lookup but immediately waits for the result.



BAD APPROACH

```
CompletableFuture<User> f =
 CompletableFuture.supplyAsync(
   () -> fetchUser(), executor);
User user = f.join();
```

What's the Problem?

- `join()` blocks the calling thread until completion.
- This can eliminate the benefits of asynchronous execution.

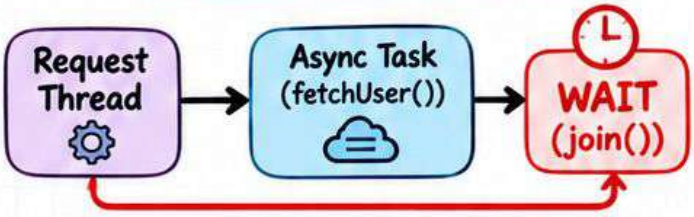
BETTER APPROACH

```
CompletableFuture<User> f =
 CompletableFuture.supplyAsync(
   () -> fetchUser(), executor);
return f.thenApply(this::toDto);
```

Key Takeaway:

Use non-blocking composition where the surrounding API supports asynchronous responses (e.g., returning a CompletableFuture or in a reactive flow).

Blocking Flow (with join())



Thread is **BLOCKED** until the async task completes.

Non-Blocking Flow (Composition)



Thread is **FREE** to do other work while the async task runs.



Interview Trap:

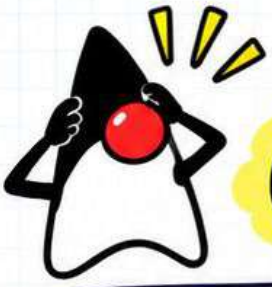
`join()` is not always wrong; blocking at the right boundary can be acceptable (e.g., in a synchronous controller, batch job or application startup).



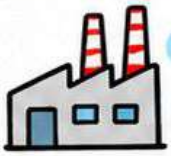
Save this for your interview! Follow @java_interview_prep

Slide 3/10

Java Interview Prep



MISTAKE #2: COMMON POOL STARVATION



Production Scenario:

Multiple services submit **blocking** HTTP calls to CompletableFuture.supplyAsync() without an executor.



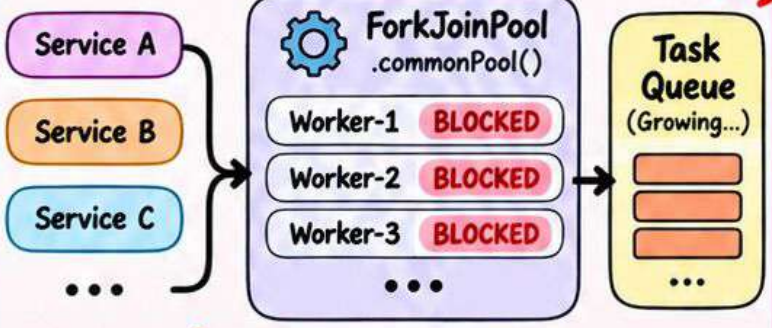
BAD APPROACH

```
CompletableFuture.supplyAsync(
    () -> remoteApi.call());
```

BETTER APPROACH

```
CompletableFuture.supplyAsync(
    () -> remoteApi.call(),
    ioExecutor);
```

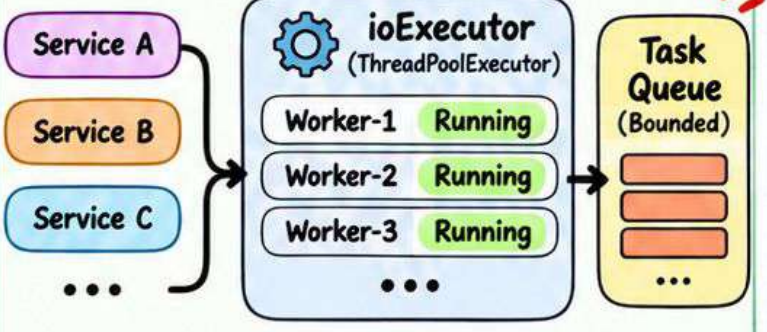
Using Common ForkJoinPool (shared by the entire JVM)



STARVATION!

Blocked I/O tasks occupy pool threads, delaying other unrelated async work.

Using a Dedicated Executor (isolated & bounded)



Isolated pool for blocking I/O. Controlled thread count and queue size.

Key Takeaways:

- Use a dedicated, bounded executor for blocking I/O (e.g., HTTP, DB, file I/O).
- Monitor executor metrics: active threads, queue depth and rejection rate.

Interview Trap:

Without an explicit executor, `supplyAsync()` normally uses the `ForkJoinPool.commonPool()`, subject to JVM fallback behavior.

Save this for your interview! Follow @java_interview_prep

Java Interview Prep

MISTAKE #3: NESTED FUTURES

Scenario: Fetch a user, then asynchronously fetch their latest order.

BAD APPROACH
(Creates a nested future)

```
CompletableFuture<
CompletableFuture<Order>>
  nested = userFuture.thenApply(
    user -> getOrderAsync(user.id())
);
```

BETTER APPROACH
(Flatten the chain)

```
CompletableFuture<Order> result =
  userFuture.thenCompose(
    user -> getOrderAsync(user.id())
  );
```

Nested Future (Harder to Use)

CompletableFuture<User> → CompletableFuture<CompletableFuture<Order>>

Nested ComplatableFuture (double wrapping)

Flattened Future (Recommended)

CompletableFuture<User> → CompletableFuture<Order>

A single future with the final result (easier to work with).

Key Rule:

- `thenApply()` = transform the result (synchronous operation).
- `thenCompose()` = chain a dependent asynchroncus operation and flatten the result.

Example Flow

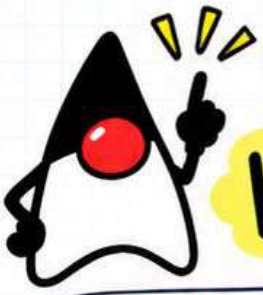
getUser Async() → getOrder Async() → CompletableFuture<Order>

Interview Trap:

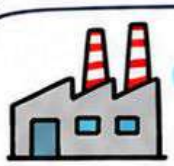
Use `thenCombine()` for independent futures, not `thenCompose()`.

Save this for your interview! Follow @java_interview_prep

Java Interview Prep



MISTAKE #4: WHERE DOES THE CALLBACK RUN?



Production Scenario:

A heavy callback unexpectedly delays a thread completing another operation.



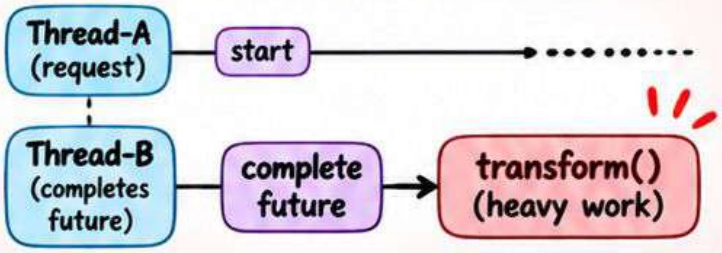
NON-ASYNC (thenApply)

```
future.thenApply(this::transform);
```

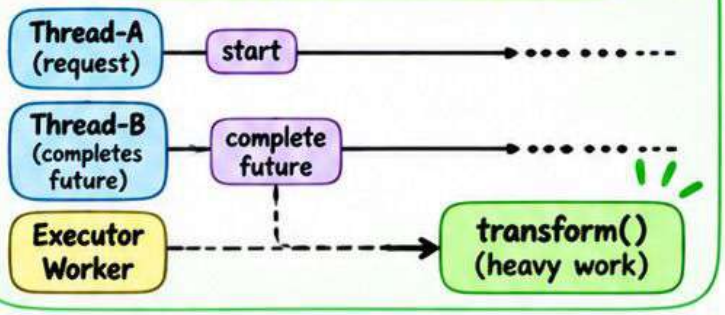
ASYNC (thenApplyAsync)

```
future.thenApplyAsync(
    this::transform, executor);
```

thenApply() – Runs on completing or registering thread



thenApplyAsync() – Runs on an executor thread



What's Happening?

- Non-async continuations (e.g., thenApply) can run on the completing thread or a thread registering thread continuation.
- Async continuations (e.g., thenApplyAsync) schedule the callback through the provided executor.

Fix / Best Practice:

- Choose the executor intentionally for expensive or blocking work.
- Use a dedicated, bounded executor (e.g., `ioExecutor`) for I/O or heavy tasks.

Interview Trap:

`thenApplyAsync()` does not guarantee a brand-new thread. It uses the given executor (or the common pool by default) where a thread may be reused.

Java Interview Prep



MISTAKE #5: SILENT ASYNC FAILURES



Production Scenario:

Payment validation **fails** inside a future, but the application never observes or handles the exceptional completion.



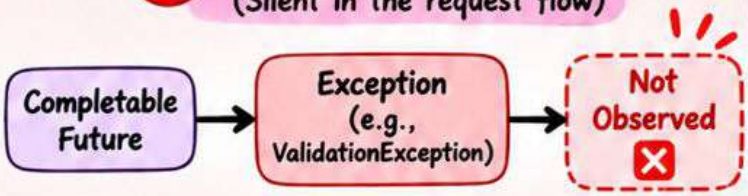
BAD APPROACH
(Ignores the failure)

```
CompletableFuture.supplyAsync(
    () -> validatePayment(),
  executor);
```

BETTER APPROACH
(Handles the failure)

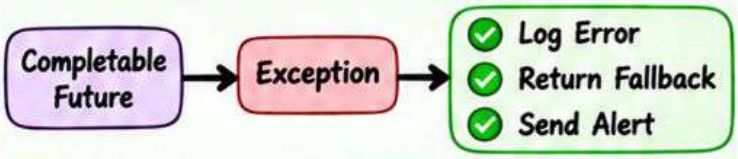
```
future.exceptionally(ex -> {
  log.error("Validation failed", ex);
  return fallback;
});
```

Unhandled Failure
(Silent in the request flow)



The failure is recorded inside the future and can go unnoticed.

Handled Failure
(Recover / Log / Alert)



Handle the failure explicitly and keep the system resilient.

Key Handlers:

`exceptionally(fn)`	→	**Recovers** from a failure (only on exception).
`handle(fn)`	→	Processes **both success and failure** (return value in both cases).
`whenComplete(fn)`	→	Observes the outcome (success or failure) for logging/metrics; **does not typically recover.**

Key Takeaway:

Always handle the exceptional completion of a CompletableFuture to avoid silent production failures.

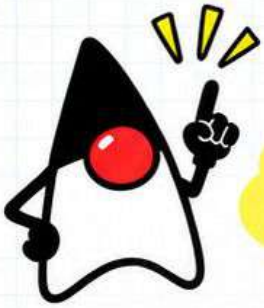
Interview Trap:

An exception inside a CompletableFuture is **not automatically thrown** on the request thread.

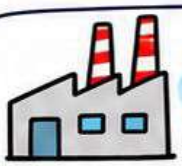
Save this for your interview! Follow @java_interview_prep

Slide 7/10

Java Interview Prep



MISTAKE #6: allOf() DOES NOT RETURN YOUR RESULTS



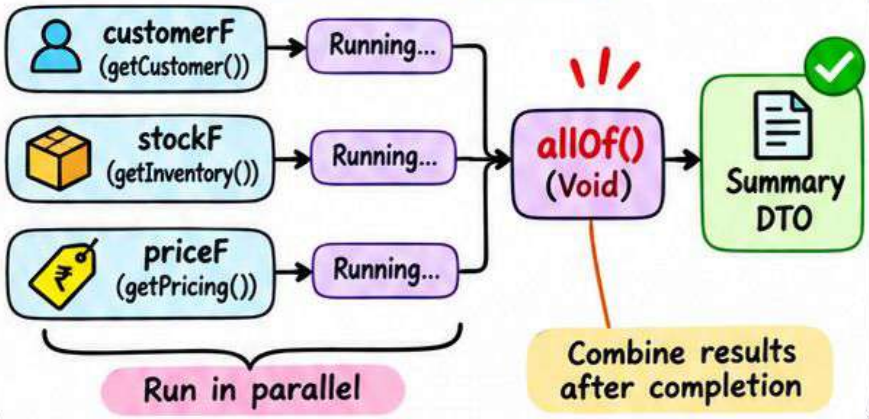
Scenario: Order API concurrently fetches customer, inventory and pricing data.



Code Example

```
CompletableFuture<Void> all =
   CompletableFuture.allOf(
      customerF, stockF, priceF
  );
all.thenApply(v -> new Summary(
     customerF.join(),
     stockF.join(),
     priceF.join()
 ));
```

How it Works



Key Takeaways:

- allOf() returns a CompletableFuture<Void>.
- It only tells you when all futures are complete.
- Collect results from the individual futures (e.g., using join() or get()) after successful completion.

What Happens on Failure?



: allOf() completes exceptionally (after all component futures finish); it does not automatically cancel the others.

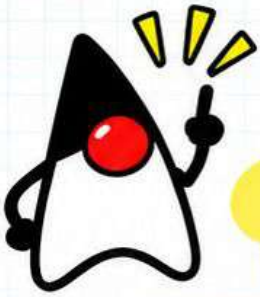
Interview Trap:

If one future fails, allOf() completes exceptionally after all component futures finish; it does not automatically cancel siblings.

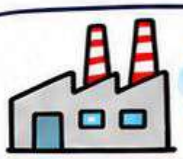
Save this for your interview! Follow @java_interview_prep

Slide 8/10

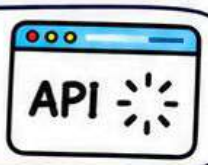
Java Interview Prep



MISTAKE #7: FUTURE TIMEOUT ≠ HTTP CANCELLATION



Scenario: A downstream API **hangs** while an Order request has a strict **deadline.**



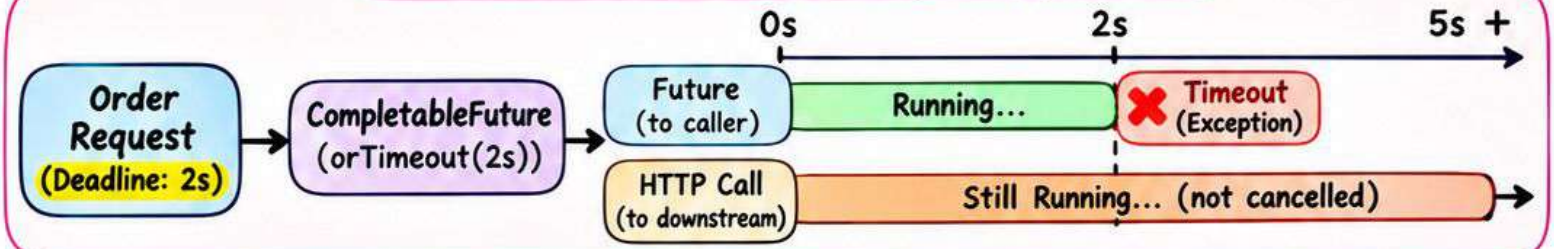
Code Example

```
CompletableFuture<Order> future =
  CompletableFuture.supplyAsync(
    () -> callDownstreamApi(),
     executor)
   .orTimeout(2, TimeUnit.SECONDS);
```

What it Means?

- **orTimeout()** completes the future exceptionally with a **TimeoutException** after the given time.
- It **does not** necessarily stop or cancel the underlying I/O (e.g., the HTTP call may still continue on the remote server).

Timeline – What Really Happens?



How to Fix It?

- Set HTTP connect / read / response timeouts (e.g., RestClient, WebClient, Feign).
- Propagate request deadlines (e.g., overall request time budget).
- Handle timeout exceptions (e.g., exceptionally(), handle()).
- Avoid assuming cancel(true) interrupts CompletableFuture work.

Key Takeaway:

- Future timeout ≠ actual cancellation of the underlying operation.
- Configure real I/O timeouts and handle the timeout at the application level.

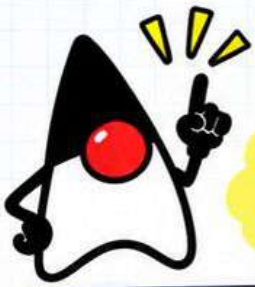
Interview Trap:

A **timed-out future** can **still leave expensive work running** (e.g., the HTTP call continues consuming resources).

Save this for your interview! Follow @java_interview_prep

Slide 9/10

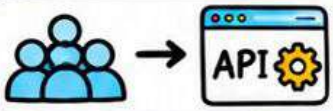
Java Interview Prep



MISTAKE #8: 10,000 ASYNC TASKS AT ONCE!



Production Scenario: A batch job calls a remote service once per customer and launches 10,000 CompletableFutures immediately.



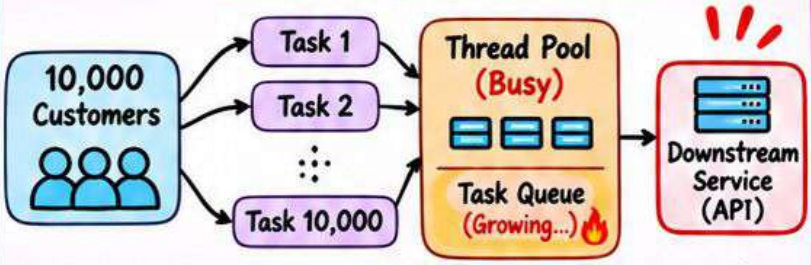
BAD APPROACH (Uncontrolled fan-out)

```
ids.stream()
  .map(id -> CompletableFuture
    .supplyAsync(() -> fetch(id),
    executor))
  .toList();
```

BETTER APPROACH (Limit concurrency)

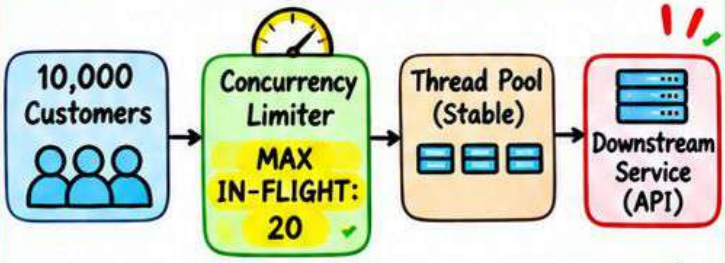
```
Semaphore limiter = new Semaphore(20);
ids.stream().map(id -> CompletableFuture
   .supplyAsync(() -> {
      limiter.acquire();
      try { return fetch(id); }
      finally { limiter.release(); }
   }, executor)).toList();
```

Uncontrolled Fan-out (Overwhelms the system)



Many concurrent calls → High load, queue buildup, timeouts & failures!

Controlled Concurrency (Safe & Stable)



Limit in-flight requests, keep the system stable and downstream healthy.

How to Fix It?

- Use a bounded executor (fixed pool with a finite queue).
- Limit in-flight requests (e.g., Semaphore / RateLimiter).
- Process in batches (e.g., 100-500 at a time).
- Add backpressure and respect downstream rate limits.

Key Takeaway:

- Don't launch thousands of futures at once.
- Control concurrency, batch requests and make it downstream-aware.

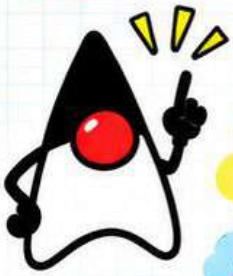
Interview Trap:

A bounded thread count does not guarantee a bounded task queue.

Save this for your interview! Follow @java_interview_prep

Slide 10/10

Java Interview Prep



CompletableFuture PRODUCTION CHEAT SHEET

8 mistakes every senior Java developer should avoid

1 Immediate join() blocking



Calling `join()/get()` right after supplyAsync() blocks the calling thread and kills concurrency.

2 Blocking work in common pool



Running blocking I/O on the `ForkJoinPool.commonPool()` can starve the pool and slow the app.

3 thenApply vs thenCompose



`thenApply` = transform a value.
`thenCompose` = chain dependent async work (flatten the future).

4 Callback execution threads



`thenApply()` may run on the completing thread, while `thenApplyAsync()` schedules via an executor.

5 Unhandled async failures



An exception inside a future can go unnoticed. Use `exceptionally()`, `handle()` or `whenComplete()`.

6 allOf result handling



`allOf()` returns CompletableFuture <Void>. Collect results from the individual futures after completion.

7 Timeouts ≠ cancellation



`orTimeout()/completeOnTimeout()` completes the future on timeout, but `does not` necessarily stop the underlying I/O.

8 Unbounded fan-out



Launching thousands of futures at once can overwhelm thread pools, queues and downstream services. `Limit concurrency` (e.g., Semaphore).

5 GOLDEN RULES

- Choose executors deliberately (e.g., separate pool for blocking I/O).
- Compose instead of blocking (use thenCompose/thenCombine).
- Handle exceptions (exceptionally/handle).
- Enforce real I/O deadlines (HTTP connect/read/response timeouts).
- Bound concurrency (Semaphore/RateLimiter/batching).

Final Interview Challenge

How would you call 3 independent services concurrently with a 2-second request deadline?



SAVE • SHARE • FOLLOW

Save this for your interview! Follow @java_interview_prep